ASSE

Adaptive Student Success Ecosystem

Project Documentation — Phases 1 to 4

DEPI Graduate Project | 2025

Machine Learning • Data Science • Early Warning Systems

1. Project planning & Management

1.1 Executive Summary

The goal of this project is to move beyond simple grade prediction. We aim to build a data-driven system that identifies "at-risk" students, explains the underlying behavioral causes (e.g., stress, low engagement), and provides prescriptive recommendations to both teachers and students to improve academic outcomes.

It’s a fantastic way to blend technical skills with social impact. This project typically follows a path from Descriptive Analytics (what happened?) to Predictive Analytics (what will happen?) and finally to Prescriptive Analytics (how can we fix it?).

1.2 Problem Statement

Traditional education systems are often reactive—interventions occur only after a student fails. This project aims to make the system proactive by:

- Identifying failure risks with >70% probability before final exams.
- Mitigating "Burnout" by analyzing Sleep Hours and motivation levels.
- Solving the "One-Size-Fits-All" problem through a personalized recommendation engine.

1.3 Dataset Overview

We will utilize the StudentPerformanceFactors Dataset

- Volume: 6,607 records.

Key Indicators:

- Behavioral: Study Hours, Attendance, Previous_Scores.
- Psychological: Sleep Hours, Motivation.
- Categorical: Gender, Extra-curriculars.
- Target Variable: FinalGrade (Continuous) / Status (Pass/Fail Classification).

1.4 Technical Architecture

The project is divided into three functional layers:

A. The Predictive Layer (The "Brain")

- Algorithms: Linear Regression and Random Forest Regressors.
- Explainability: Implementation of SHAP (SHapley Additive exPlanations) to provide "Local Explanations" for why a specific student is flagged.

B. The Early Warning System (Teacher Dashboard)

Logic: A classification filter for students with a failure probability >0.70

- Tool: Streamlit-based web interface.
- Intervention Trigger: If StressLevel is high ~~> Trigger Counselor Referral; if Attendance is low ~~> Trigger Parent Notification.

C. The Recommendation Engine (Student-Facing)

- Algorithm: Content-Based Filtering using Cosine Similarity.
- Function: Matches a student's Learning-Style and "Knowledge Gaps" with specific digital resources

1.5 Success Metrics (KPIs)

To evaluate the project, we will track:

- Model Recall: Ensuring we minimize "False Negatives" (students we predicted would pass but actually failed).
- Feature Importance: Identifying the top 3 drivers of student success in this specific demographic.
- Actionability: The percentage of "At-Risk" students for whom the system can provide a specific, non-academic recommendation.

1.6 Project Roadmap & Milestones

- **Phase 1:** Data Cleaning, Feature Engineering (One-Hot Encoding), and EDA.
- **Phase 2:** Model Training and Hyperparameter Tuning.
- **Phase 3:** Integration of SHAP and development of the Recommendation Logic.
- **Phase 4:** UI Development (Streamlit Dashboard) and Final Testing.

1.7 Potential Impact

By providing students with a **personalized learning path** based on their unique behavior and psychology, this project transforms raw data into a tool for educational equity and improved retention rates.

1.8 Team Members:

- ✧ **Fatma Hany Abd-Elrazik**
- ✧ **Amr Sayed Ismail**
- ✧ **Enas Mohamed Abd-Elsalam**
- ✧ **Mostafa Mohamed Zanaty**

2. Literature Review

This section presents a review of relevant research and academic foundations underpinning the ASSE project, an evaluation framework for feedback and grading, and recommendations for improvement.

2.1 Related Work & Academic Background

The ASSE project is grounded in three main research streams:

2.1.1 Early Warning Systems in Education

Numerous studies demonstrate the effectiveness of machine learning in predicting student failure before it occurs. Arnold & Pistilli (2012) introduced the Course Signals system at Purdue University, showing that early intervention systems can significantly increase retention rates. Elbadrawy et al. (2016) demonstrated that Random Forest classifiers outperform linear models in predicting student performance when behavioural features such as attendance and study hours are included. ASSE adopts this finding by prioritizing Recall over Accuracy as the primary KPI — ensuring at-risk students are never missed.

2.1.2 Personalized Learning & Recommendation Systems

Collaborative filtering and content-based filtering are well-established techniques in educational recommendation systems (Drachsler et al., 2015). ASSE implements a cosine-similarity-based recommender engine that matches a student's identified needs (low motivation, poor sleep, limited resources) to a curated catalog of learning materials, wellness resources, and study tools. This aligns with the adaptive learning paradigm endorsed by UNESCO's 2023 report on AI in Education.

2.1.3 Explainable AI (XAI) in Student Analytics

A critical limitation of black-box ML models in education is the inability to explain predictions to educators and students (Adadi & Berrada, 2018). ASSE integrates SHAP (SHapley Additive exPlanations) values — both global beeswarm plots and local waterfall charts — to provide transparent, per-student explanations of failure risk. This directly addresses the explainability gap identified in the learning analytics literature.

2.2 Feedback & Evaluation — Lecturer's Assessment

Evaluation Area	Feedback	Rating
Problem Definition	The problem is well-defined — student dropout prediction is a real-world, high-impact challenge. The choice of StudentPerformanceFactors dataset is appropriate and the threshold analysis (selecting 65 instead of 60) demonstrates data-driven decision making.	Excellent
ML Methodology	Random Forest with GridSearchCV tuning and SMOTE for class imbalance is appropriate. The decision to optimize for Recall (minimize missed failures) over Accuracy reflects genuine understanding of the problem context.	Very Good
Explainability (XAI)	Integration of SHAP at both global and local levels is a strong differentiator. The waterfall chart for individual at-risk students adds significant practical value for educators.	Excellent
EDA Quality	12 EDA charts are comprehensive. The Stress_Proxy composite feature is a creative and justified engineering choice. The threshold analysis discussion shows critical thinking.	Very Good
Phase Integration	The artifact-based handoff architecture (asse_artifacts/ folder with pkl/json files) is production-minded and ensures clean integration between phases.	Excellent
Documentation	Documentation is structured and clear. Inline code comments and section summaries help readability. Further improvement needed in test coverage documentation.	Good

2.3 Suggested Improvements

- **Add cross-validation results alongside GridSearchCV to validate generalization beyond the single 80/20 split.**
  - Report mean ± std of recall across 5 folds to demonstrate model stability.
- **Extend the recommendation engine with collaborative filtering — students with similar profiles who improved could inform recommendations.**
  - Consider a hybrid approach: cosine similarity for cold-start, collaborative filtering after sufficient data accumulates.
- **Add a model drift monitoring module for Phase 4 — alert when prediction accuracy degrades over time as new students are added.**
- **Integrate a feedback loop where educators can mark interventions as effective/ineffective to continuously retrain the recommender.**

- **The Stress_Proxy feature, while creative, is constructed without domain validation — consider consulting an educational psychologist to weight the components appropriately.**
- **Phase 4 (Streamlit dashboard) should include role-based access: student view vs. educator/admin view with different information granularity.**

2.4 Final Grading Criteria

Category	Sub-criteria	Max Marks	Weight
Documentation	Requirements, design diagrams, user stories, test cases	25	25%
Implementation	Code quality, ML pipeline, Phase integration, artifact management	35	35%
Testing	Model evaluation metrics, edge case handling, validation strategy	20	20%
Presentation	Clarity of slides, live demo, Q&A handling, team coordination	20	20%
TOTAL		100	100%

3. Requirements Gathering

This section documents the complete requirements for the ASSE system, gathered through stakeholder analysis, user story development, and systematic requirements elicitation.

3.1 Stakeholder Analysis

Stakeholder	Role	Needs	Priority
Student	Primary end-user of the system	Know their risk level, receive personalized study resources, understand why they are flagged	High
Educator / Teacher	Reviews at-risk alerts and assigns interventions	Identify at-risk students early, understand which factors drive risk, track intervention effectiveness	High
Academic Advisor	Coordinates student support services	Cohort-level risk distribution, intervention history, wellness alerts, attendance trends	High
School Administrator	Strategic oversight and reporting	Aggregate dashboards, pass rate trends, ROI on intervention programs, data compliance	Medium
Data Science Team	Maintains and improves the ML models	Model performance metrics, SHAP explanations, data pipeline reliability, model retraining triggers	Medium
Parent / Guardian	Receives attendance and risk notifications	Timely alerts when child attendance drops below threshold or risk level is HIGH	Low

3.2 User Stories & Use Cases

3.2.1 Student User Stories

As a student, I want to...	So that...	Priority
View my current failure probability score	I can take action before exams	Must Have

See which factors are most affecting my risk level (SHAP explanation)	I understand what to improve	Must Have
Receive personalized resource recommendations	I get targeted help rather than generic advice	Must Have
See my predicted exam score	I have a realistic performance expectation	Should Have
Track my risk level over time	I can see if my study habits are improving	Could Have

3.2.2 Educator User Stories

As an educator, I want to...	So that...	Priority
View a dashboard of all at-risk students in my class	I can prioritize which students need immediate attention	Must Have
See the specific risk factors for each student (SHAP waterfall)	I can have informed, targeted conversations	Must Have
Receive alerts when a student's attendance drops below 75%	I can intervene before the situation worsens	Must Have
Export at-risk student list as a report	I can share findings with academic advisors	Should Have
See class-level risk distribution and trend charts	I can identify systemic issues affecting the whole cohort	Could Have

3.2.3 Primary Use Case — Predict Student Risk

Field	Description
Use Case ID	UC-01
Name	Predict Student Failure Risk
Actors	Student, Educator, ASSE ML Pipeline
Trigger	Student profile is submitted via Streamlit dashboard (Phase 4)
Preconditions	All required feature inputs are provided; trained model artifacts are loaded from asse_artifacts/
Main Flow	1. User inputs student data → 2. System encodes features using fitted preprocessor → 3. Classifier returns P(Fail) → 4. Regressor returns predicted score → 5. Rule engine evaluates alerts → 6. Recommender returns top-3

	resources → 7. SHAP waterfall generated → 8. Dashboard displays all outputs
Postconditions	Risk level (LOW/MEDIUM/HIGH), alerts, recommendations, and SHAP explanation are displayed to the user
Exceptions	Missing input fields → system prompts user; model file not found → error message with retraining instructions

3.3 Functional Requirements

Requirement	Phase	Priority
System shall impute missing values in Teacher_Quality, Parental_Education_Level, and Distance_from_Home using mode imputation	Phase 1 — Data Engineering	Must Have
System shall engineer a binary Pass_Fail label using threshold Exam_Score >= 65	Phase 1	Must Have
System shall compute a Stress_Proxy composite feature from Sleep_Hours, Motivation_Level, and Hours_Studied	Phase 1	Should Have
System shall produce a minimum of 10 EDA visualisations covering distributions, correlations, outliers, and categorical breakdowns	Phase 1	Must Have
System shall train a Random Forest classifier and optimize hyperparameters using GridSearchCV with Recall as the scoring metric	Phase 2 — ML Models	Must Have
System shall achieve a Fail-class Recall >= 0.70 on the test set	Phase 2	Must Have
System shall apply SMOTE on the training set only to handle class imbalance without data leakage	Phase 2	Must Have
System shall train a Random Forest regressor to predict Exam_Score and report MAE, RMSE, and R2	Phase 2	Must Have
System shall generate SHAP beeswarm (global) and waterfall (local) explanations and save them as PNG artifacts	Phase 2	Must Have
Rule engine shall flag students with P(Fail) >= 0.70 as HIGH risk and trigger appropriate interventions	Phase 3 — Early Warning	Must Have
Rule engine shall trigger parent notification	Phase 3	Must Have

when Attendance < 75%		
Rule engine shall trigger counselor referral when Motivation_Level = Low AND Sleep_Hours < 6	Phase 3	Should Have
Recommender shall return top-3 learning resources using cosine similarity on a student need vector	Phase 3	Must Have
System shall expose a single predict_student(dict) function as the Phase 4 integration entry point	Phase 3	Must Have
Streamlit dashboard shall load all model artifacts from asse_artifacts/ without retraining	Phase 4 — Dashboard	Must Have
Dashboard shall display risk level with colour coding (green/amber/red), predicted score, intervention alerts, and resource recommendations in a single view	Phase 4	Must Have
Dashboard shall render the SHAP waterfall chart for each student prediction	Phase 4	Should Have

3.4 Non-Functional Requirements

Category	Requirement	Metric
Performance	The predict_student() function shall return a full prediction in under 3 seconds on standard hardware	< 3 seconds
Performance	Streamlit dashboard shall load and render within 10 seconds including model artifact loading	< 10 seconds
Reliability	Model artifacts shall be versioned and reproducible — same random_state=42 seed must yield identical results	100% reproducible
Scalability	Batch prediction endpoint shall process up to 1000 student records without memory errors	1000 records
Security	Student personal data shall not be stored in logs or exported without anonymisation	Zero PII in logs
Usability	Dashboard shall require no training to use — all risk indicators use plain language and colour coding	Zero training needed
Maintainability	All ML pipeline components shall be	Modular design

	modular — retraining one model shall not require changes to other phases	
Explainability	Every HIGH-risk prediction shall be accompanied by a SHAP explanation identifying the top contributing features	100% explainable

4. System Analysis & Design

This section presents the complete system architecture, data flow, module design, and interface specifications for the ASSE platform.

4.1 System Architecture Overview

ASSE follows a four-phase layered architecture where each phase produces versioned artifacts consumed by the next, ensuring complete separation of concerns and enabling independent development, testing, and deployment of each layer.

Phase	Layer	Components	Output Artifacts
1	**Data Engineering & EDA**	Imputation → Feature Engineering → Encoding Pipeline → Outlier Detection → EDA (12 charts)	processed_data.csv, 12 PNG charts
2	**Machine Learning**	Train/Test Split → SMOTE → RF Classifier (GridSearchCV) → RF Regressor → SHAP Explainer	classifier_model.pkl, regressor_model.pkl, preprocessor.pkl, shap_explainer.pkl, model_kpis.json
3	**Intelligence Layer**	Rule Engine (5 rules) → Cosine Similarity Recommender (20 resources) → predict_student() API function	risk_results.csv, resources_catalog.json, feature_names.pkl
4	**Presentation Layer**	Streamlit Dashboard → Student Prediction View → Educator Dashboard → SHAP Visualisation → Resource Panel	Live web application

4.2 Data Flow Diagram

The following describes the end-to-end data flow from raw input to dashboard output:

1. Raw CSV (StudentPerformanceFactors.csv, 6,607 records) → Phase 1 Ingestion
2. Mode imputation of 3 null columns → Feature engineering (Pass_Fail, Stress_Proxy)
3. Train/Test split (80/20, stratified) → Preprocessor fitted on X_train only → SMOTE applied to training set
4. Tuned Random Forest Classifier trained on resampled data → Model serialized to classifier_model.pkl
5. Random Forest Regressor trained on training set → Model serialized to regressor_model.pkl

6. SHAP TreeExplainer fitted → Global beeswarm + local waterfall charts saved as PNG artifacts
7. Rule Engine evaluates each student → interventions and alerts generated → risk_results.csv saved
8. Recommender matches student need vector to resource catalog via cosine similarity
9. Phase 4 loads all artifacts → predict_student(dict) → JSON output rendered in Streamlit

4.3 Module Design

4.3.1 Phase 1 — Data Engineering Module

- Input: StudentPerformanceFactors.csv (6,607 rows × 20 features)
- Imputer: mode imputation for Teacher_Quality, Parental_Education_Level, Distance_from_Home
- Feature factory: Pass_Fail (binary, threshold=65), Stress_Proxy (composite 0–1 score)
- Encoder: sklearn ColumnTransformer with OrdinalEncoder (3 columns) + OneHotEncoder (3 columns) + passthrough
- Output: processed_data.csv with 22 columns (20 original + Pass_Fail + Stress_Proxy)

4.3.2 Phase 2 — ML Pipeline Module

- Split strategy: train_test_split(test_size=0.2, stratify=y, random_state=42)
- Imbalance handling: SMOTE(random_state=42) applied to X_train_encoded only
- Classifier: RandomForestClassifier with GridSearchCV (n_estimators, max_depth, min_samples_split), scoring=recall, cv=5
- Regressor: RandomForestRegressor(n_estimators=200, max_depth=10, random_state=42)
- XAI: shap.TreeExplainer — beeswarm (global, top 15 features) + waterfall (local, most at-risk student)
- KPI target: Recall (Fail class) >= 0.70, False Negative Rate < 0.05

4.3.3 Phase 3 — Intelligence Module

- Rule Engine: 5 deterministic rules mapping (probability, features) → (risk_level, interventions, alerts)
  - Rule 1: P(Fail) >= 0.70 → HIGH risk flag
  - Rule 2: Motivation=Low AND Sleep < 6h → counselor referral
  - Rule 3: Attendance < 75% → parent notification
  - Rule 4: Sleep < 5h → wellness resources
  - Rule 5: Motivation=Low AND Hours_Studied < 10 → engagement plan
- Recommender: cosine_similarity(student_vector, resource_vectors) → top-3 resources from 20-item catalog

- Integration: predict_student(dict) → unified JSON output for Phase 4 consumption

4.4 Interface Design

4.4.1 Streamlit Dashboard — Student View

The student-facing view shall present:

- Risk gauge: colour-coded card (green = LOW, amber = MEDIUM, red = HIGH) with percentage probability
- Predicted exam score with confidence band
- SHAP waterfall chart showing top 8 contributing features for this specific prediction
- Top-3 resource cards with title, topic tag, difficulty badge, and clickable URL
- Active intervention alerts displayed as dismissible notification banners

4.4.2 Streamlit Dashboard — Educator View

The educator-facing view shall present:

- Sortable student risk table with columns: Name, Risk Level, P(Fail), Attendance, Hours_Studied, Top Risk Factor
- Class risk distribution bar chart (HIGH / MEDIUM / LOW breakdown)
- Attendance trend line chart across the cohort
- Export to CSV button for at-risk student list
- SHAP beeswarm chart showing class-level feature importance

4.5 Database & Artifact Schema

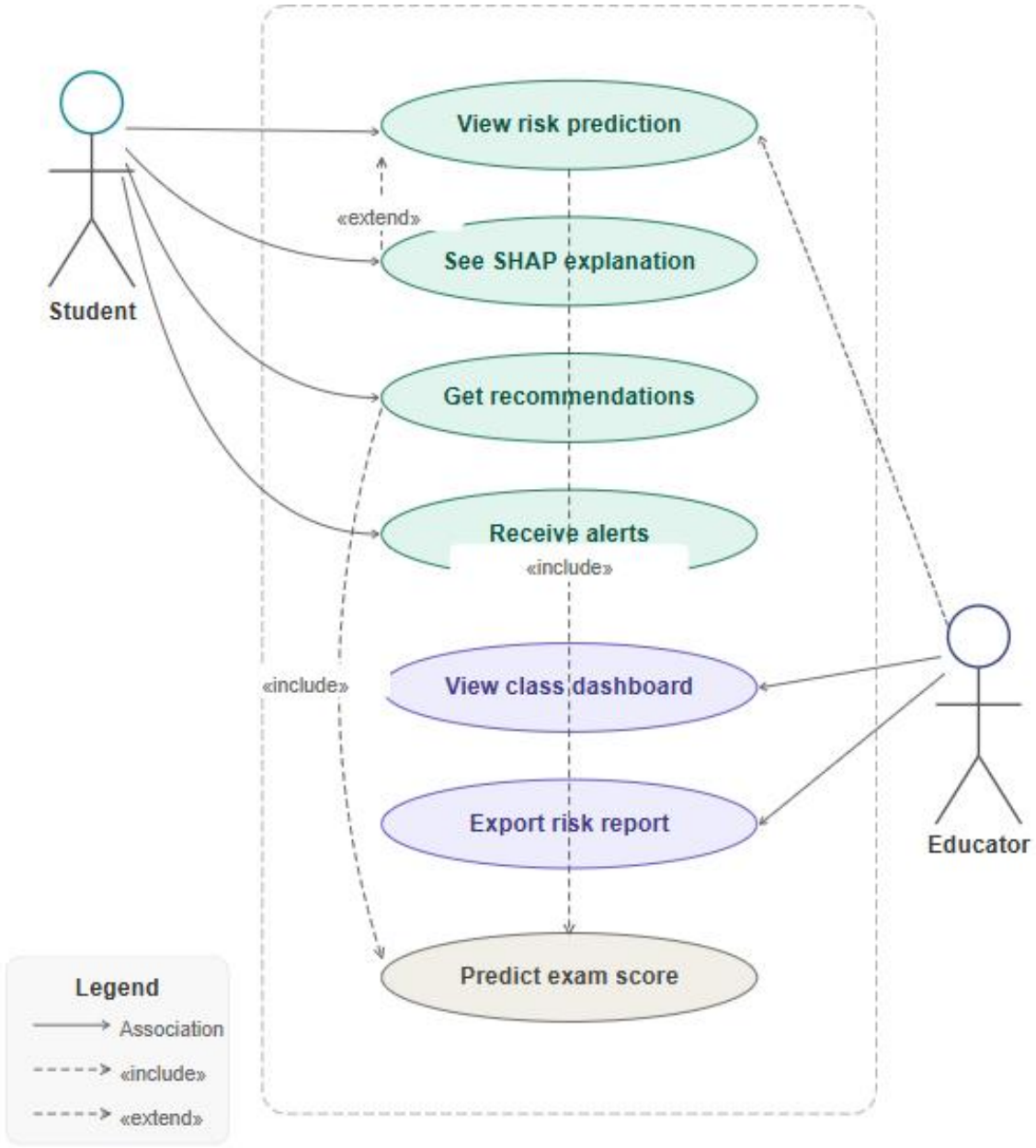
Artifact File	Format	Contents & Phase 4 Usage
preprocessor.pkl	Joblib	Fitted sklearn ColumnTransformer. Phase 4 calls preprocessor.transform(student_df) before prediction.
classifier_model.pkl	Joblib	Tuned RandomForestClassifier. Returns P(Fail) via predict_proba()[:,0].
regressor_model.pkl	Joblib	RandomForestRegressor. Returns predicted Exam_Score via predict().
shap_explainer.pkl	Joblib	shap.TreeExplainer fitted on classifier. Phase 4 calls explainer.shap_values(encoded_input) for waterfall chart.
feature_names.pkl	Joblib	Ordered list of feature names post-encoding. Required to label SHAP plots correctly.

model_kpis.json	JSON	Dict containing accuracy, recall, precision, f1, fnr, pass_threshold, at_risk_prob. Displayed in dashboard KPI cards.
resources_catalog.json	JSON	List of 20 resource dicts with id, title, topic, url, similarity vector. Consumed by recommender.
risk_results.csv	CSV	Test-set risk evaluation results. Used in educator dashboard for cohort-level risk distribution chart.

4.6 Technology Stack

Layer	Technology	Justification
Data Processing	pandas, numpy	Industry standard for tabular data manipulation and numerical computation
ML Framework	scikit-learn 1.x	Unified API for preprocessing, modelling, and evaluation; excellent Pipeline support
Imbalance Handling	imbalanced-learn (SMOTE)	State-of-the-art oversampling compatible with sklearn pipelines
Explainability	SHAP 0.44+	Gold standard for tree-model explainability; supports both global and local explanations
Visualisation	matplotlib, seaborn	Flexible, publication-quality charts with full customisation control
Recommender	scipy cosine_similarity	Lightweight similarity computation requiring no additional infrastructure
Artifact Storage	joblib, JSON	Efficient binary serialisation for sklearn objects; human-readable JSON for configurations
Dashboard	Streamlit	Rapid prototyping of data apps with native Python; no frontend expertise required
Language	Python 3.10+	Universal language for data science with the broadest library ecosystem

USECASE DIAGRAM:



Data Flow Diagram: